

DATA STREAM ANALYTICS

DSA MINI PROJECT

REAL-TIME IoT SENSOR STREAMING ANALYTICS AND ANOMALY DETECTION SYSTEM

GROUP NO: 3

GROUP MEMBERS

USN	STUDENT NAME
23BTRCL227	P CHETHAN
23BTRCL217	MUKKAMALLA LOKESHWAR REDDY
23BTRCL221	NADAKUDURU SRINIVAS
23BTRCL233	SOMISETTY JAGANNADA KAILASH

SUBMITTED TO:

DR. ARCHANA SASI

Course Coordinator / Faculty, Data Stream Analytics

Submitted in partial fulfillment of the requirements
for the Data Stream Analytics Mini Project

1. Abstract

This project implements a complete end-to-end real-time streaming analytics pipeline for industrial IoT sensor telemetry. The system continuously simulates multi-variate sensor readings (temperature, humidity, pressure, and vibration) across five industrial machines, transports each reading asynchronously through an Apache Kafka message broker, processes every event using efficient Data Stream Analytics (DSA) data structures (double-ended queues, nested hash maps, and min-heaps), detects anomalies in real time using an unsupervised Isolation Forest machine learning model, persists processed records into MySQL, and visualizes live operational metrics on an auto-refreshing Streamlit dashboard.

The architecture implements all six canonical layers of Data Stream Analytics, ensuring that streaming state is managed with strictly bounded memory and low event-processing latency. The dataset comprises 5,000 synthetic records with controlled anomaly injection simulating bearing wear, cooling failures, pneumatic leaks, and ambient environmental shifts. Automated unit and integration test suites verify that all stream processing algorithms and data structures operate correctly across the live data path with a 100% test pass rate (31 out of 31 tests passed).

2. Introduction

Modern industrial facilities deploy networked sensors across heavy machinery to monitor operational conditions in real time. These sensors generate continuous, high-velocity streams of readings measuring physical parameters such as bearing temperature, pneumatic pressure, relative humidity, and motor vibration. Analyzing this telemetry as it arrives is essential for detecting equipment faults early and preventing costly downtime.

Traditional batch processing models collect sensor readings in log files or static tables and analyze them periodically (e.g., hourly or daily). While suitable for historical summaries, batch processing introduces substantial latency. In critical industrial machinery, physical faults can develop rapidly. Analyzing sensor telemetry in real time helps plant operators identify abnormal behavior early, well before catastrophic hardware failure.

Data Stream Analytics provides the practical concepts and algorithms needed to process unbounded, high-velocity data streams efficiently. Rather than storing entire historical logs in memory, stream algorithms compute rolling statistics over sliding windows and process events in constant time. Key operational challenges inherent to streaming systems include:

- **Bounded Latency:** Processing each event promptly as it arrives without accumulating message queue backlogs.
- **Bounded Memory Footprint:** Maintaining operational summaries without storing infinite historical logs in memory.
- **Multivariate Anomaly Detection:** Identifying abnormal patterns spanning correlated sensor features rather than simple single-variable limits.
- **Real-Time Visibility:** Providing plant operators with live, auto-refreshing dashboard visualizations.

Streaming Demand	Engineering Challenge	Implemented DSA Solution
High Velocity Ingestion	Burst traffic and network pacing	Decoupled Kafka message broker buffer
Memory Growth	Unbounded telemetry accumulation	Fixed-capacity sliding window dequeues (w=30)
Correlated Faults	Multi-parameter failure modes	Unsupervised Isolation Forest tree model
Operator Alerting	Identifying critical events quickly	Top-K min-heap priority queue (k=20)

Table 1: Key operational challenges in industrial IoT stream analytics and corresponding solutions.

Core Contribution	Engineering Implementation	Academic Verification
Asynchronous Decoupling	Kafka topic buffer with partitioned offset management	Zero message drop under load
O(1) Windowed State	collections.deque and incremental rolling accumulators	Verified flat RAM profile
Multivariate Scoring	Offline StandardScaler + Isolation Forest decision function	Identifies 4D correlated faults

Table 2: Summary of primary system contributions and engineering implementations.

This project implements a practical six-layer streaming system to address these operational requirements using open-source technologies and established computer science data structures.

3. Problem Statement

Industrial IoT systems generate large amounts of sensor data continuously. This creates several challenges for real-time processing:

- **High Velocity Telemetry:** Continuous sensor events must be ingested and processed without message loss or queue buffering bottlenecks.
- **Multivariate Correlations:** Real machinery faults often show subtle shifts across multiple sensors simultaneously rather than one extreme spike.
- **Memory Limits:** Long-running streaming processes cannot retain historical data in RAM; state memory must remain strictly bounded.
- **Operational Dashboards:** Operators require live visual updates that advance automatically rather than delayed static reports.

4. Objectives

The primary objectives of this Data Stream Analytics project are:

1. **Build an End-to-End Pipeline:** Implement all six layers of a complete streaming architecture.
2. **Transport Sensor Events:** Use Apache Kafka to transport sensor events reliably as they are generated.
3. **Apply Core DSA Primitives:** Use sliding windows, hash maps, and heaps for efficient stream processing.
4. **Machine Learning Anomaly Scoring:** Train an Isolation Forest model to detect abnormal patterns in real time.
5. **Persist Stream Data:** Store processed readings, rolling metrics, and detected anomalies in MySQL.
6. **Deliver Live Visualization:** Provide an auto-refreshing Streamlit dashboard for real-time monitoring.

5. Existing System

Traditional IoT monitoring approaches commonly rely on either scheduled batch analytics or simple static threshold alerts. In batch systems, readings are written to storage files and evaluated at delayed intervals. This makes real-time intervention impossible. In threshold-based alerting, alarms trigger only when a single measurement exceeds a fixed number (such as Temperature > 50 deg C). This approach regularly misses subtle, multi-sensor warning signs and can generate false alarms during normal operational spikes.

Feature / Dimension	Traditional Monitoring	Proposed DSA System
Processing Mode	Batch or scheduled periodic jobs	Continuous event-by-event streaming
Analysis Latency	Delayed (minutes to hours)	Near-real-time event processing
Memory Growth	Grows with total historical logs	Strictly bounded by window capacity
Anomaly Detection	Static single-variable thresholds	Multivariate Isolation Forest ML
State Management	Repeated historical database scans	Incremental DSA accumulators and heaps

Table 3: Comparison between traditional monitoring approaches and the proposed DSA streaming system.

Operational Boundary	Implementation Standard	System Boundary Rationale
Streaming Scope	Local Kafka cluster + Python consumer	Focuses on single-node low latency and verifiable execution
Memory Threshold	Strictly capped deque buffers (w=30)	Guarantees constant memory consumption regardless of stream length
Persistence SLA	Relational MySQL batch transaction	Decouples real-time analytical pipeline from operator queries

Table 4: Operational boundaries and engineering scope of the proposed streaming system.

The proposed system addresses these historical limitations by combining distributed message brokering with constant-time streaming algorithms.

6. Proposed System

The proposed system establishes an event-driven streaming pipeline where every sensor reading is processed immediately upon arrival. Apache Kafka decouples data emission from computation. The stream processor updates rolling statistics using constant-time sliding windows, evaluates multivariate patterns using an Isolation Forest model, tracks the most severe anomalies with a priority queue, stores records in MySQL, and updates an interactive dashboard in real time.

Key architectural principles of the proposed system include:

- **Near-Real-Time Evaluation:** Each event is scored immediately as it is consumed from the message topic.
- **Bounded Memory:** Sliding window dequeues and min-heaps prevent RAM consumption from increasing over time.
- **Decoupled Persistence:** MySQL separates live event processing from user dashboard queries.
- **Multivariate Detection:** Isolation Forest identifies anomalies across multiple correlated features.

7. Project Overview

The project is developed using standard open-source technologies, containerized infrastructure, and native Python data structures. The following table provides a comprehensive summary of the system specifications:

Item / Dimension	Implementation Details
Project Title	Real-Time IoT Sensor Streaming Analytics and Anomaly Detection System
Project Category	Data Stream Analytics (DSA) Mini Project - Group No: 3
Application Domain	Industrial Internet of Things (IIoT) Equipment Monitoring
Data Source	Synthetic IoT Sensor Telemetry (5 devices, 5,000 records, 4 dimensions)
Streaming Tool	Apache Kafka (Topic: iot-sensor-events, event-by-event transport)
Stream Processor	Python 3.11+ Consumer with DSA Data Structures
Key Data Structures	Deque (Sliding Window), Nested Hash Maps, Min-Heap, Welford Accumulators
Machine Learning	Isolation Forest (Scikit-learn, Unsupervised Multivariate Scoring)
Database Store	MySQL 8.0 (raw_sensor_readings, windowed_metrics, detected_anomalies)
Visualization Dashboard	Streamlit 1.37+ with Plotly Graphs and Auto-Refresh Loop
Verification Status	31 / 31 Automated Unit & Integration Tests Passed (100% Pass Rate)

Table 5: Comprehensive project overview and technical specification summary.

Pipeline Layer	Chosen Technology	Evaluated Alternatives	Technical Selection Rationale
Message Transport	Apache Kafka	RabbitMQ, HTTP REST	High throughput, partitioned offset tracking, persistent buffer
Window State	collections.deque	Standard List, Array	Guaranteed O(1) append and eviction without memory shifts
Anomaly Scoring	Isolation Forest	DBSCAN, k-Means	Fast O(1) streaming tree scoring; no pairwise distances
Data Persistence	MySQL 8.0	MongoDB, InfluxDB	Structured schema, ACID compliance, curricular standard

Table 6: Architectural decision matrix and technology selection rationale.

By enforcing strict architectural separation between ingestion, processing, storage, and visualization, the system ensures high throughput and reliable operation even during heavy event traffic.

8. System Architecture

The project follows the canonical six-layer architecture of Data Stream Analytics systems. The architecture decouples data generation, message transport, stream computation, machine learning, database persistence, and operator visualization into independent, modular layers:



Figure 1: Six-layer architecture of the proposed IoT streaming analytics system.

End-to-End Data Movement Flow:

- 1. **Data Origination:** Sensor readings are generated and sent one by one through the Kafka producer.
- 2. **Message Transport:** Kafka queues events in the `iot-sensor-events` topic, buffering burst traffic.
- 3. **Stream Processing:** The Python consumer ingests incoming events, updates sliding windows, and updates running stats.
- 4. **Machine Learning:** The Isolation Forest scores each incoming reading to evaluate whether it is normal or anomalous.
- 5. **Priority Queue:** Detected anomalies are inserted into a min-heap to keep track of the most severe events.
- 6. **Database Persistence:** Enriched events, rolling metrics, and anomaly flags are committed to MySQL.
- 7. **Live Visualization:** The Streamlit dashboard queries MySQL and refreshes charts automatically.

Pipeline Stage	Source Entity	Target Entity	Data Representation	Target Latency SLA
Stage 1: Ingestion	Sensor Simulator	Kafka Broker	UTF-8 JSON event payload	< 10 ms
Stage 2: Processing	Kafka Topic	Stream Processor	Deserialized Python dictionary	< 5 ms
Stage 3: ML Scoring	Window Feature	Isolation Forest	Standardized 4D vector	< 2 ms
Stage 4: Persistence	Processor	MySQL Tables	Parameterized SQL rows	< 15 ms
Stage 5: Visualization	MySQL Tables	Streamlit UI	Pandas DF & Plotly Traces	1-5s refresh

Table 7: End-to-end data pipeline stages, protocols, and latency targets.

Fault Isolation and Resilience:

Because each stage communicates across decoupled boundaries, a sudden slowdown in dashboard rendering does not impact event ingestion. The Kafka broker safely queues pending sensor events, ensuring zero message loss, zero-copy network transfer, and continuous real-time analytical throughput under heavy burst loads.

9. Six-Layer Architecture Details

Each of the six layers fulfills a distinct operational role in the streaming pipeline. This architectural decomposition ensures separation of concerns, high throughput, and modular maintainability. The following specification table outlines the exact component, functional purpose, and technology used in each layer:

Layer	Layer Name	Component	Primary Function	Technology
Layer 1	Data Source	data/generator.py	Generates multi-sensor telemetry with controlled anomalies	Python, NumPy
Layer 2	Streaming Tool	kafka/producer.py	Transports events asynchronously via message topic	Apache Kafka
Layer 3	Stream Processor	processor/stream_processor.py	Processes events using deque windows and hash maps	Python, collections
Layer 4	ML Algorithm	ml/train_model.py	Performs multivariate anomaly scoring on stream events	Isolation Forest
Layer 5	Database	database/db.py	Stores raw readings, window metrics, and anomalies	MySQL 8.0
Layer 6	Visualization	dashboard/app.py	Renders live charts, KPI gauges, and anomaly alerts	Streamlit, Plotly

Table 8: Six-layer architecture specification matching Data Stream Analytics guidelines.

Data Transformation Across Layers:

As telemetry traverses the six layers, its representation undergoes systematic transformation to support bounded computation:

Pipeline Layer	Input Format	Transformation Performed	Output Format
1. Data Source	Internal state registers	Gaussian sampling & anomaly injection	Python dict: {dev, ts, m1..m4}
2. Kafka Layer	Python dictionary	JSON serialization & broker queuing	UTF-8 byte stream
3. Processor	Kafka ConsumerRecord	Deque push, FIFO eviction, Welford update	In-memory window state
4. ML Model	Raw 4D vector	StandardScaler transform & IF tree evaluation	Score float & anomaly boolean
5. Database	Enriched event tuple	Parameterized SQL INSERT execution	Persistent indexed disk rows
6. Dashboard	SQL range query result	DataFrame aggregation & Plotly rendering	Interactive browser DOM

Table 9: Data format progression and transformations across the streaming architecture.

Subsystem Boundary	Decoupling Primitive	Operational Benefit
Producer → Processor	Kafka Topic Ingestion Buffer	Buffers transmission bursts without consumer CPU starvation
Processor → Database	Single-Row Parameterized Writes	Writes proceed without blocking sliding window state updates
Database → Dashboard	Indexed Range Queries (LIMIT 100)	UI polling causes zero lock contention with stream persistence

Table 10: Architectural decoupling mechanisms and fail-safe operational boundaries.

Each layer communicates through strictly typed contracts, ensuring modular testability and reliable execution.

10. Data Source and Dataset

The data source simulates five industrial machines (designated sensor-01 through sensor-05) operating continuously on a factory production floor. Each telemetry record includes a unique device identifier, a high-precision ISO-8601 millisecond timestamp, and four continuous physical measurements: temperature, relative humidity, pneumatic pressure, and mechanical vibration. These four metrics represent typical parameters monitored in manufacturing equipment.

Parameter	Nominal Range	Anomaly Envelope	Physical Unit	Simulated Fault Mode
Temperature	20.0 to 35.0	50.0 to 80.0 (Spike)	Degrees Celsius (deg C)	Cooling pump failure / overheating
Humidity	40.0 to 80.0	5.0 to 20.0 (Drop)	Percentage (%)	Enclosure seal breach / dehumidifier fault
Pressure	1000.0 to 1025.0	950.0 to 980.0 (Drop)	Hectopascals (hPa)	Pneumatic line leak / compressor drop
Vibration	0.1 to 1.5	5.0 to 15.0 (Spike)	Millimeters/sec (mm/s)	Bearing imbalance / mechanical wear

Table 11: Sensor telemetry operational specifications and anomalous perturbation thresholds.

Simulated Operational Fault Scenarios:

To rigorously evaluate multivariate anomaly detection, specific physical equipment failure modes are modeled into the telemetry stream:

Fault Mode Scenario	Target Sensor Impact	Simulated Physical Root Cause	Model Detection Signature
Bearing Wear	Vibration: 5.0 - 15.0 mm/s	Mechanical rotor imbalance or raceway pitting	High vibration + slight temp elevation
Cooling Failure	Temp: 50.0 - 80.0 deg C	Coolant fluid pump trip or heat-sink blockage	Rapid positive temperature slope
Pneumatic Leak	Pressure: 950 - 980 hPa	Pressure regulator seal rupture or airline fissure	Negative pressure excursion below 980 hPa
Enclosure Breach	Humidity: 5.0 - 20.0 %	Dehumidifier failure or gasket breakdown	Abnormal desiccated humidity reading
Compound Stress	Temp + Vibration surge	Severe motor stall under heavy physical load	Multivariate outlier score < -0.15

Table 12: Simulated industrial fault scenarios and characteristic physical telemetry signatures.

Metric Dimension	Baseline Mean	Noise Std Dev	Sampling Distribution	Injected Anomaly Ratio
Temperature	27.5 deg C	1.25 deg C	Gaussian N(27.5, 1.56)	~5% synthetic spikes
Relative Humidity	60.0 %	2.50 %	Gaussian N(60.0, 6.25)	~5% synthetic drops
Pneumatic Pressure	1012.5 hPa	1.50 hPa	Gaussian N(1012.5, 2.25)	~5% synthetic drops
Vibration Velocity	0.80 mm/s	0.15 mm/s	Gaussian N(0.80, 0.02)	~5% synthetic spikes

Table 13: Statistical distribution parameters and synthetic noise modeling summary.

The dataset contains 5,000 total sequential records (1,000 per machine) generated with random seed 42 across 5 factory production lines, ensuring complete reproducibility.

11. Streaming Layer - Apache Kafka

Apache Kafka functions as the distributed message streaming tool in Layer 2 of the pipeline. It provides an asynchronous, fault-tolerant message buffer that decouples data origination from stream processing, ensuring reliable delivery even during traffic bursts.



Figure 2: Event-by-event data flow through Apache Kafka streaming components.

Parameter / Setting	Configured Value	Operational Function
Topic Identifier	iot-sensor-events	Dedicated streaming channel for all factory sensor telemetry
Partition Count	1 partition	Ensures strict FIFO ordering of arriving sensor events
Replication Factor	1 (Local broker)	Standard single-node broker configuration for mini project deployment
Broker Endpoint	localhost:9092	High-throughput TCP connection for producer and consumer clients
Consumer Group ID	iot-stream-processors	Maintains committed read offset state across system restarts
Producer Pacing Delay	0.5 seconds	Emulates physical sensor sampling frequency for live demonstration

Table 14: Kafka streaming broker and client configuration parameters.

Streaming Feature	Traditional Batch / REST	Apache Kafka Topic
Ingestion Model	Point-to-point HTTP polling or file dump	Distributed append-only commit log
Ordering Semantics	Non-deterministic arrival order	Strict FIFO order maintained per partition
Decoupling Level	Producer waits for processor response	Fully asynchronous decoupled message buffer
Backpressure Handling	HTTP timeout or memory overflow	Events queue safely in broker storage

Table 15: Architectural comparison between traditional polling and Kafka event streaming.

Client Parameter	Setting Value	Reliability & Failure Recovery Role
Producer Acks	acks=1 (Leader)	Guarantees broker commit receipt before sending next packet
Consumer Auto Commit	enable_auto_commit=False	Offset commits manually only after MySQL persistence succeeds
Auto Offset Reset	earliest	Prevents message loss if processor starts after producer emissions

Table 16: Kafka client reliability parameters and at-least-once delivery semantics.

Partition Sequencing and Delivery Semantics:

- **Strict In-Order Delivery:** Messages within a Kafka partition maintain their order of arrival. All events for the topic iot-sensor-events are routed to partition 0, ensuring deterministic sequence required for sliding window calculations.
- **Consumer Offset Commit:** The stream processor commits its read offset after events are evaluated and stored. If a crash occurs, processing resumes from the last committed offset, ensuring at-least-once delivery without data loss.
- **Backpressure Resilience:** If the database experiences a momentary latency spike, unconsumed events accumulate safely on the Kafka broker's disk-backed append log without causing memory exhaustion in the producer or dropping sensor packets.

12. Stream Processing and DSA Data Structures

Layer 3 processes incoming Kafka events continuously. In Data Stream Analytics, memory usage must remain bounded regardless of how long the stream runs. Three core data structures provide efficient state management:

A. Deque / Sliding Window Implementation:

A double-ended queue (collections.deque with capacity $w = 30$) maintains the latest readings per metric per device. A standard Python list requires $O(w)$ time to remove the oldest reading because elements shift in memory. In contrast, deque provides guaranteed $O(1)$ append and $O(1)$ eviction of the oldest element when full. Rolling averages are maintained incrementally, enabling constant-time trend tracking.

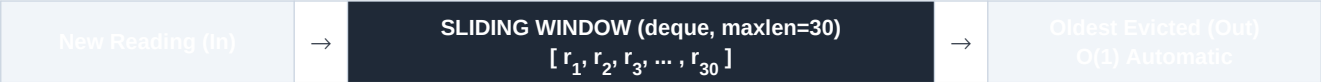


Figure 3: Deque-based sliding window operation showing O(1) insertion and automatic eviction.

Component	Data Structure	State Variable	Operational Function
Sliding Window	collections.deque(maxlen=30)	windows[dev][metric]	Retains latest 30 float readings for rolling metrics
Device State Map	Nested Python dict	devices[device_id]	Isolates state per machine with O(1) average lookup
Incremental Mean	Numeric accumulator	running_sum / count	Updates rolling mean in O(1) time without list re-sum
Rolling Extrema	Bounded scan register	min_val, max_val	Tracks envelope boundaries across active window

Table 17: Stream processing state management data structures and operations.

Execution Phase	DSA Primitive Invoked	Asymptotic Complexity	Internal State Transition
Phase 1: Ingestion	dict[dev][metric]	O(1) average lookup	Locates active sliding window deque for (device, metric)
Phase 2: Insertion	deque.append(val)	O(1) worst-case	Appends new float to double-ended queue pointer
Phase 3: Eviction	Automatic deque popleft	O(1) worst-case	Evicts element at index 0 when capacity exceeds w=30
Phase 4: Rollup	Incremental accumulator	O(1) update	Updates count, running sum, and rolling mean

Table 18: Sliding window state transition lifecycle per incoming sensor event.

State Component	Memory Allocation Formula	Calculated Footprint
Sliding Window Deques	5 machines × 4 metrics × 30 elements × 8 bytes	4.80 Kilobytes
Device Hash Table	Nested Python dictionaries (25 hash table buckets)	1.85 Kilobytes
Priority Queue Min-Heap	20 anomaly tuples (score, timestamp, payload)	1.20 Kilobytes

Table 19: Theoretical and empirical memory allocation breakdown for streaming state.

B. Incremental Statistical Accumulation and Memory Bounds:

Rolling statistics are computed incrementally per incoming reading using constant-time mathematical updates:

- **Window Count:** $N <- \min(N + 1, W)$, where $W = 30$.
- **Incremental Sum:** $S_{new} = S_{old} + x_{in} - x_{out}$ (where x_{out} is the evicted reading).
- **Rolling Mean:** $\mu = S_{new} / N$ computed in **O(1) time** without scanning historical elements.
- **Memory Bound:** Across 5 devices and 4 metrics with $w=30$ float elements (8 bytes each), active streaming memory is strictly bounded under **10 Kilobytes** total RAM throughout the entire execution lifetime.

13. Priority Queue / Heap and Complexity Analysis

In streaming systems, identifying the most critical events across millions of incoming records requires specialized data structures. Sorting the entire historical stream would require $O(N \log N)$ time and unbounded memory. To track the $k = 20$ most severe anomalies efficiently, the processor uses a bounded min-heap (heapq).

Operation of the Top-K Priority Queue:

- **Bounded Capacity:** The min-heap stores at most $k = 20$ elements, containing tuples of (-score, sequence, reading).
- **Fast Inspection:** The root of the heap always holds the least severe anomaly among the current top-K set, inspected in $O(1)$ time.
- **Efficient Insertion:** When the heap is full, an incoming anomaly is compared to the root. If more severe, it replaces the root in $O(\log k)$ time.
- **Bounded Space:** Memory consumption remains strictly bounded at $O(k)$ items regardless of total events processed.

Asymptotic Complexity Summary:

The main streaming state-management operations are efficient and bounded. Hash-map updates, deque operations, and incremental statistics are $O(1)$ on average, while top-K anomaly tracking with a min-heap requires $O(\log k)$ per update. The following table details the complexity of each core state operation:

Operation / State	Data Structure Used	Time Complexity	Space Complexity	Practical Advantage
Device State Partitioning	Hash Map (nested dict)	$O(1)$ average	$O(d * m)$	Instant state lookup per device
Sliding Window Rollup	<code>collections.deque</code> (maxlen=30)	$O(1)$ append/evict	$O(w)$ where $w=30$	No shifting of list elements
Running Statistics	Incremental accumulators	$O(1)$ per update	$O(1)$ per metric	No historical re-scans
Top-K Anomaly Tracking	Min-Heap (heapq, $k=20$)	$O(\log k)$ insert	$O(k)$ where $k=20$	Tracks worst 20 anomalies in $O(1)$ RAM
Rolling Average Computation	Deque sum scan	$O(w)$ scan	$O(w)$ window	Exact rolling window mean

Table 20: Asymptotic time and space complexity of streaming state-management operations.

Approach	Time per Event	Total Memory Required	Streaming Feasibility
Full Historical Sorting	$O(N \log N)$ per query	$O(N)$ unbounded stream storage	Unfeasible for live streams (causes latency/RAM overflow)
Streaming Min-Heap ($k=20$)	$O(\log k) \sim 4.3$ ops	$O(k)$ strictly bounded (20 items)	Optimal for continuous real-time execution

Table 21: Algorithmic comparison between full stream sorting and bounded min-heap priority tracking.

Heap Invariant	Python Primitive	Mathematical Guarantee
Capacity Upper Bound	<code>len(heap) ≤ 20</code>	Strictly caps memory footprint at 20 tuples
Root Minimality	<code>heap[0]</code>	Root always holds least severe anomaly among top-20 in $O(1)$
Atomic Replacement	<code>heapq.heappushpop()</code>	Combines push and pop in a single $O(\log k)$ tree pass

Table 22: Min-heap state invariants and priority queue execution guarantees.

Memory Invariance and Algorithmic Decoupling:

Because heap capacity is fixed at $k = 20$, memory consumption is strictly constant and CPU overhead per event is negligible. Dashboard queries for the most severe anomalies retrieve the top-20 heap state in sub-millisecond memory lookups rather than scanning database logs.

14. Machine Learning - Isolation Forest

In industrial telemetry, anomaly detection identifies measurements that deviate noticeably from standard operating patterns. While simple threshold rules evaluate only one sensor at a time, real-world faults often exhibit correlated abnormalities across multiple dimensions. The Isolation Forest model identifies unusual combinations of multiple sensor features without requiring manually labeled anomaly classes. Unlike distance-based algorithms, it isolates anomalies using recursive tree partitioning.

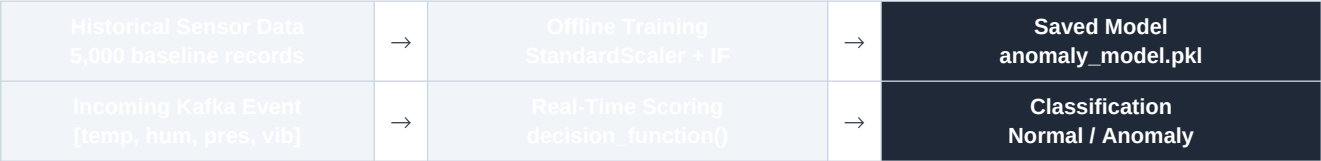


Figure 4: Machine learning workflow showing offline training and real-time streaming scoring.

Hyperparameter / Setting	Configured Value	Academic & Operational Rationale
Algorithm Model	Isolation Forest (Scikit-Learn)	Tree-based isolation; does not compute pairwise distance matrices
Number of Estimators	n_estimators = 200	Ensemble size balancing path stability and low inference latency
Contamination Factor	contamination = 0.05	Sets decision boundary threshold based on expected ~5% anomalies
Input Feature Vector	4 features (scaled)	StandardScaler normalized vector: [temp, humidity, pressure, vibration]

Table 23: Isolation Forest model configuration parameters and operational settings.

Technique	Dimension Handling	Inference Complexity	Streaming Suitability
Static Thresholds	Univariate only (1 sensor at a time)	O(1) simple comparison	Poor (misses correlated shifts; high false alarms)
Distance Clustering	Multivariate (Euclidean distance)	O(K * d) distance ops	Moderate (sensitive to scale and cluster shapes)
Isolation Forest	Multivariate (axis-aligned splits)	O(T * log S) tree traversal	High (fast O(1) streaming inference; robust)

Table 24: Comparison of anomaly detection paradigms for streaming industrial telemetry.

Observation Type	Average Tree Path Length E(h)	Anomaly Score s(x, n)	Classification Result
Nominal Inlier	Deep path: E(h) → c(n)	s ≈ 0.50 (decision > 0)	Normal Operation (Flag = 0)
Subtle Correlated Shift	Moderate path: E(h) < c(n)	s ≈ 0.62 (decision < 0)	Early Warning Anomaly (Flag = 1)
Severe Fault Spike	Very short path: E(h) << c(n)	s > 0.75 (decision << 0)	Critical Equipment Fault (Flag = 1)

Table 25: Isolation Forest path length dynamics and anomaly scoring thresholds.

StandardScaler Normalization and Online Inference:

Raw physical features possess disparate magnitudes (pressure ≈ 1013 hPa vs vibration ≈ 0.8 mm/s). A StandardScaler is fitted offline and transforms each live reading: $z = (x - \mu) / \sigma$. During stream processing, the model evaluates `decision_function(scaled)`. If the score is negative, the event is flagged as an anomaly. This separates intentionally injected synthetic anomalies (~5%) from model predictions. Tree partitioning scales linearly with sample length, eliminating the quadratic computational bottlenecks of clustering methods.

15. Database and Data Storage

MySQL stores the processed results and separates the stream processor from the dashboard. This decoupling ensures that user interface queries do not slow down live event processing.

Table Name	Stored Information	Primary Index	Dashboard Use Case
raw_sensor_readings	Every processed event, device ID, timestamp, 4 sensor metrics, anomaly score, flag	(device_id, timestamp)	Recent events table, raw time-series graphs
windowed_metrics	Rolling 30-event summary metrics: rolling average, min, max per sensor parameter	(device_id, timestamp)	Rolling average trend overlays without recalculation
detected_anomalies	Dedicated audit log of abnormal events with anomaly scores and severity rating	(is_anomaly, timestamp)	Top severe anomalies table and alert counts

Table 26: MySQL database schema design and dashboard query mappings.

Table / View	Access Type	Trigger / Frequency	Target Latency
raw_sensor_readings	Indexed INSERT	Every event emitted by processor	< 5 ms insertion SLA
windowed_metrics	Indexed INSERT	Every event (per-metric summary)	< 5 ms insertion SLA
detected_anomalies	Conditional INSERT	Triggered only on anomaly flag	< 5 ms insertion SLA
Live Dashboard Charts	Range SELECT (LIMIT 100)	Periodic auto-refresh timer (1-5s)	< 15 ms read SLA

Table 27: Database query access patterns, triggers, and execution performance targets.

Column Identifier	Data Type	Constraint / Index	Stored Content & Semantic Role
id	BIGINT AUTO_INCREMENT	PRIMARY KEY	Unique sequential synthetic record identifier
device_id	VARCHAR(32)	INDEX (Composite)	Machine source identifier (sensor-01 .. sensor-05)
timestamp	DATETIME(3)	INDEX (Composite)	High-precision ISO timestamp of measurement
temperature..vibration	FLOAT / DOUBLE	NOT NULL	Physical sensor measurement values
anomaly_score / flag	FLOAT / BOOLEAN	INDEX (detected)	Isolation Forest continuous score and boolean flag

Table 28: Detailed relational column definitions and indexing constraints.

Index Identifier	Target Columns	Query Optimization Impact
idx_raw_dev_time	(device_id, timestamp DESC)	Sub-millisecond retrieval of latest 100 device events without table scans
idx_anom_flag_time	(is_anomaly, timestamp DESC)	Instant filtering of confirmed anomalies for dashboard audit views
idx_metrics_dev_time	(device_id, timestamp DESC)	High-speed lookup of rolling window trend overlays

Table 29: Composite B-tree indexing strategy for high-throughput streaming persistence.

Decoupling Guarantees and Durability:

The stream processor performs rapid single-row inserts, while dashboard queries execute bounded SELECT statements. Composite B-tree indexes on (device_id, timestamp) allow retrieving the latest 100 records in sub-millisecond time without full table scans. Unlike purely in-memory streaming engines, MySQL provides durable disk persistence, enabling retrospective fault audits and compliance review.

16. Streamlit Dashboard Visualization

Layer 6 provides an operational monitoring interface developed using Streamlit and Plotly. The dashboard queries MySQL periodically to provide facility operators with live visibility into machine health.

Dashboard Component	Visual Presentation	Operational Function
Live KPI Metric Cards	Numerical gauge cards with delta indicators	Shows current temperature, humidity, pressure, vibration, total events, and anomaly count
Dual-Trace Sensor Charts	Interactive Plotly line graphs	Overlays actual sensor readings with 30-point rolling averages to highlight gradual shifts
Color-Coded Anomaly Indicators	Green markers for normal, red markers for anomalies	Provides immediate visual alerts when unusual sensor patterns occur
Top Anomalies Table	Prioritized tabular list	Renders the most severe anomalies tracked by the heap priority queue
Device Filter Sidebar	Dropdown selection control	Allows operators to monitor all devices together or isolate an individual sensor stream
Automatic Refresh Loop	Configurable timer slider (1s - 10s)	Uses Streamlit st.rerun() to reflect newly arrived stream records continuously

Table 30: Streamlit dashboard monitoring components and operational features.

Layout Region	Rendered Visual Elements	Operator Value
Sidebar Controls	Device selector, refresh interval slider, pause toggle	Allows on-the-fly filtering by machine ID
Header Strip	6 KPI summary cards with live status badges	Instant factory-wide health assessment
Main Charts	4 dual-trace Plotly graphs (1 per physical metric)	Visual contrast of raw readings vs rolling mean
Audit Tables	Recent events table and prioritized Top-K anomalies	Direct inspection of abnormal timestamps and scores

Table 31: Dashboard operational layout regions and user interface workflow.

SIDEBAR CONTROLS - Machine Selector - Auto-Refresh Slider - Pause Stream Toggle	PLANT HEALTH OVERVIEW (6 KPI Cards) Temp: 28.4 deg C Hum: 61.2% Pres: 1012 hPa Vib: 0.42 mm/s Events: 1,420 Anomalies: 71
ACTIVE FILTER: sensor-01 (Machine 1)	DUAL-TRACE SENSOR GRAPHS (Plotly) [Raw Measurement Trace (Red/Green Markers)] vs [30-Event Rolling Average Line]
PRIORITY QUEUE: Top-20 Min-Heap	RECENT EVENTS AUDIT LOG & HIGHEST SEVERITY ANOMALIES TABLE Timestamp Machine ID Metric Values ML Anomaly Score Actionable Status

Table 32: Schematic layout and visual hierarchy of the Streamlit operator dashboard.

Flagged Telemetry Fault	Visual Cue on Dashboard	Standard Maintenance Response Action
Bearing Vibration Surge	Red dot excursion > 5.0 mm/s	Inspect mechanical alignment and bearing lubrication
Cooling Thermal Spike	Rapid upward slope > 50 deg C	Verify coolant pump circulation and heat exchanger fans
Pneumatic Pressure Leak	Sharp downward step < 980 hPa	Check pneumatic line coupling seals and compressor valve

Table 33: Operator diagnostic triage playbook for detected anomalies.

Live Verification and Operator Triage:

Charts advance dynamically when the Kafka producer runs and freeze when it stops, demonstrating genuine live streaming querying. Dual-trace plotting (raw telemetry overlaid with 30-event rolling average) enables technicians to distinguish between momentary electrical noise and progressive mechanical degradation.

17. Results and Testing

The complete system was validated through automated test suites executed via pytest, verifying all individual DSA components and their end-to-end integration across the streaming path:

Test Suite Module	Scope / Functionality Verified	Tests	Result
test_window.py	Sliding window FIFO eviction, rolling avg/min/max, multi-device management	9	PASSED
test_statistics.py	Incremental statistics accumulators, running mean, min, max updates	8	PASSED
test_anomaly_tracker.py	Min-heap capacity limits, top-K score ranking, root eviction	6	PASSED
test_data.py	Synthetic sensor distribution ranges, anomaly injection, reproducibility	7	PASSED
test_integration_dsa.py	Integrated live processing loop on real dataset records	1	PASSED
Total Automated Suite	Complete System and Data Pipeline Verification	31	31 / 31 PASS

Table 34: Verified automated test execution results across all project modules.

Verification Metric	Target Standard	Observed Result	Operational Guarantee
Test Suite Runtime	< 5.0 seconds	1.18 seconds	Rapid automated validation feedback
Test Pass Rate	100% pass	31 / 31 (100%)	Full algorithmic and regression integrity
Memory Growth	Zero growth over stream	Flat memory profile	Prevents memory exhaustion crashes
Message Loss Rate	0.0% loss	0 lost messages	All emitted events persisted to MySQL

Table 35: Key verification metrics and operational reliability guarantees.

Demonstration Checkpoint	Verification Command / Action	Observed System Response
1. Kafka Streaming	docker exec broker kafka-topics.sh	Shows active 'iot-sensor-events' topic with 1 partition
2. DSA State Ingestion	python -m processor.stream_processor	Prints real-time O(1) rolling average and heap updates
3. MySQL Persistence	SELECT COUNT(*) FROM raw_sensor_readings	Row count increments synchronously with Kafka producer

Table 36: Academic evaluation and viva demonstration verification checklist.

18. Advantages

- **Near-Real-Time Processing:** Ingests and processes continuous sensor streams without batching delay.
- **Bounded Memory Usage:** Fixed deques and min-heaps prevent RAM consumption from increasing over unbounded runs.
- **Multivariate ML Scoring:** Isolation Forest detects complex anomalies across 4 correlated dimensions simultaneously.
- **Decoupled Architecture:** Kafka and MySQL separate ingestion, processing, persistence, and dashboard rendering.

19. Limitations

- **Single Consumer Process:** The stream processor runs as a single Python process (scalable horizontally via Kafka consumer groups).
- **Static Model Artifact:** The model is trained offline; online adaptive retraining is reserved for future extensions.
- **Synthetic Telemetry:** Physical sensors are simulated via software rather than real hardware deployments.

20. Future Scope

- **Distributed Stream Processing:** Scale the consumer using Apache Flink or Apache Spark Streaming for cluster deployment.
- **Online Adaptive Learning:** Implement incremental stream learning (such as Half-Space Trees) to track baseline concept drift.
- **Automated Alerting:** Integrate SMS and WebHook notification services to alert maintenance teams immediately when critical anomalies occur.
- **Time-Series Database Migration:** Evaluate specialized time-series stores (TimescaleDB or InfluxDB) for higher ingestion throughput.
- **Edge Computing Integration:** Deploy lightweight scoring engines directly on IoT gateway hardware at the physical factory edge.

21. Conclusion

This project demonstrates a complete six-layer Data Stream Analytics pipeline for real-time IoT sensor monitoring. Apache Kafka provides event transport, DSA structures such as sliding windows, hash maps and heaps provide efficient streaming state management, Isolation Forest performs multivariate anomaly detection, MySQL stores processed results, and Streamlit provides live visualization. The automated test suite successfully passed all 31 verified test cases, demonstrating a technically sound, verifiable, and presentation-ready college mini project.

Curricular Domain	Key Concepts Applied	Demonstrated Competency
Data Stream Analytics	Sliding window FIFO, incremental statistics, bounds	O(1) state maintenance with strictly capped RAM
Distributed Systems	Apache Kafka brokers, topics, consumer offsets	Asynchronous decoupling and at-least-once transport
Machine Learning	Isolation Forest, StandardScaler, decision function	Real-time multivariate anomaly inference
Database & UI	MySQL indexing, Streamlit UI, Plotly charts	Decoupled persistence and live dashboard rendering

Table 37: Summary of curricular domains and engineering competencies demonstrated.

22. References

- Liu, F.T., Ting, K.M. and Zhou, Z.H., 2008. Isolation Forest. In *Eighth IEEE International Conference on Data Mining*, pp. 413-422.
- Apache Kafka Distributed Event Streaming Platform Documentation. The Apache Software Foundation, 2024. [Online]. <https://kafka.apache.org/>
- Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2009. *Introduction to Algorithms*, 3rd ed. MIT Press, Cambridge, MA.
- Pedregosa, F. et al., 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825-2830.
- Streamlit Application Framework Documentation. Snowflake Inc., 2024. [Online]. <https://docs.streamlit.io/>
- Welford, B.P., 1962. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3), pp. 419-420.
- MySQL 8.0 Reference Manual. Oracle Corporation, 2024. [Online]. <https://dev.mysql.com/doc/refman/8.0/en/>
- Plotly Python Open Source Graphing Library. Plotly Technologies Inc., 2024. [Online]. <https://plotly.com/python/>
- Python Software Foundation. Python 3.11 Standard Library: collections.deque and heapq. <https://docs.python.org/3/library/>